FRED TALKS

20th October 2026

CONTENTS

Control the ideas, not the code
ANTIREZ.COM · 1245 WORDS

A statistical approach to model evaluations
ANTHROPIC.COM · 1598 WORDS

Big tech engineers need big egos
SEANGOEDECKE.COM RSS FEED · 1975 WORDS

Zen of AI Coding
YOAV AVIRAM · 1912 WORDS

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · SOURCE

Look at the past history of this blog. There are many blog posts abou

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even *not* accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

Matteo Collina yesterday asked me, in reply to my tweet: but didn’t y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:

A statistical approach to model evaluations

ANTHROPIC.COM · 14 SEP 2025 · SOURCE

Suppose an AI model outperforms another model on a benchmark of interest—testing its general knowledge, for example, or its ability to solve computer-coding questions. Is the difference in capabilities real, or could one model simply have gotten lucky in the choice of questions on the benchmark?

With the amount of public interest in AI model evaluations—informally called “evals”—this question remains surprisingly understudied among the AI research community. This month, we published a new research paper that attempts to answer the question rigorously. Drawing on

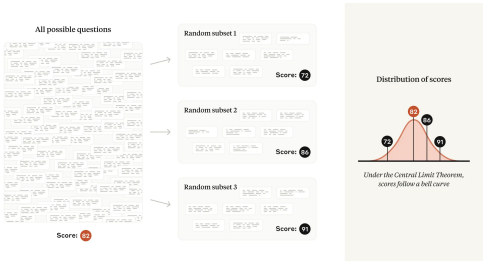
statistical theory and the experiment design literature, the paper makes a number of recommendations to the AI research community for reporting eval results in a scientifically informative way. In this post, we briefly go over the reporting recommendations, and the logic behind them.

Recommendation #1: Use the Central Limit Theorem

Evals often consist of hundreds or thousands of unrelated questions. MMLU, for instance, contains questions as diverse as:

- Who discovered the first virus?
- What is the inverse of $f(x)=4-5x$?
- Who said that “Jurisprudence is the eye of law”?

To compute an overall eval score, each question is separately scored, and then the overall score is (usually) a simple average of these question scores. Typically, researchers focus their attention on this observed average. But in our paper, we argue that the real object of interest should not be the *observed* average, but rather the *theoretical* average across all possible questions. So if we imagine that eval questions were drawn from an unseen “question universe,” we can learn about the average score in that universe—that is, we can measure the underlying *skill*, independent of the “luck of the draw”—using statistical theory.



If we imagine that eval questions were drawn from a “question universe,” then eval scores will tend to follow a normal distribution, centered around the average score of all possible questions.

This formulation buys us analytic robustness: if a new eval were to be created with questions having the same difficulty distribution as the original eval, we should generally expect our original conclusions to hold.

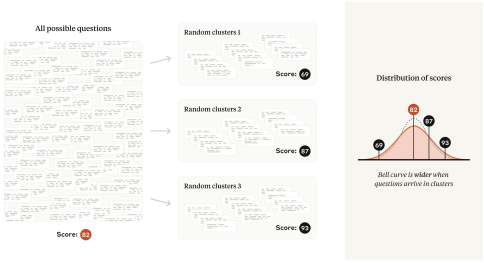
In technical terms: under the fairly mild conditions of the Central Limit Theorem, the mean values of several random samples taken from the same underlying distribution will tend to follow a normal distribution. The standard deviation (or width) of that normal distribution is commonly known as the standard error of the mean, or SEM. In our paper, we encourage researchers to report the SEM, derived from the Central Limit Theorem, alongside each calculated eval score—and we show researchers how to use the SEM to quantify the difference in theoretical means between two models. A 95% confidence interval can be calculated from the SEM by adding and subtracting 1.96 × SEM from the mean score.

Recommendation #2: Cluster standard errors

Many evals violate the above assumption of independently selected questions, and instead consist of groups of closely related questions. For example, several questions in a reading-comprehension eval may ask about the same passage of text. Popular evals that follow this pattern include DROP, QuAC, RACE, and SQuAD.

For these evals, each question's selection from the "question universe" is no longer independent. Because including several questions about the same passage of text will yield less information than selecting the same number of questions about different passages of text, a naive application of the Central Limit Theorem to the case of non-independent questions will lead us to underestimate the standard error—and potentially mislead analysts into drawing incorrect conclusions from the data.

Fortunately, the problem of clustered standard errors has been extensively studied in the social sciences. When the inclusion of questions is non-independent, we recommend clustering standard errors on the unit of randomization (for example, passage of text), and we provide applicable formulas in our paper.



If questions arrive in related clusters—a common pattern in reading-comprehension evals—eval scores will be more spread-out compared to the non-clustered case.

In practice, we have found that clustered standard errors on popular evals can be over three times as large as naive standard errors. Ignoring question clustering may lead researchers to inadvertently detect a difference in model capabilities when in fact none exists.

Recommendation #3: Reduce variance within questions

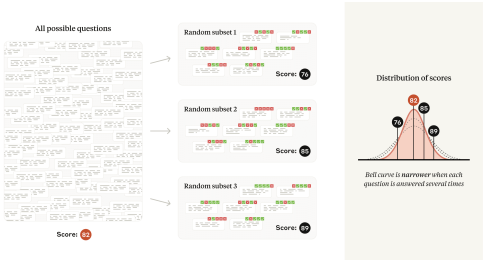
Variance is a measurement of how spread-out a random variable is. The variance of an eval score is the square of the standard error of the mean, discussed above; this quantity depends on the amount of variance in the score on each individual eval question.

A key insight of our paper is to decompose a model's score on a particular question into two terms that are added together:

- The mean score (the average score that the model would achieve if asked the same question an infinite number of times—even if the model might produce a different answer each time); and
- A random component (the difference between a realized question score and the mean score for that question).

Thanks to the law of total variance, reducing the variance in the random component directly leads to a smaller standard error of the overall mean, and thus greater statistical precision. Our paper highlights two strategies for reducing variance in the random component depending on whether or not the model is asked to think step by step before answering (a prompting technique known as CoT, or chain-of-thought reasoning).

If an eval uses chain-of-thought reasoning, we recommend resampling answers from the same model several times, and using the question-level averages as the question scores fed into the Central Limit Theorem. We note that the Inspect framework correctly computes standard errors in this way via its *epochs* parameter.



If a model produces answers non-deterministically, then generating (and grading) several answers per question will result in less spread-out eval scores.

If the eval does not use chain-of-thought reasoning (i.e., its answers are not “path dependent”), we note that the random component in the score may often be eliminated altogether using next-token probabilities from the language model. For example, if the correct answer to a multiple-choice question is “B”, we would simply use the probability of the model producing the token “B” as the question score. We are not aware of an open-source evals framework which implements this technique.

Recommendation #4: Analyze paired differences

Eval scores don’t have any meaning on their own; they only make sense in relation to one another (one model outperforms another model, or ties another model, or outperforms a person). But could a measured difference between two models be due to the specific choice of questions in the eval, and randomness in the models’ answers? We can find out with a two-sample *t*-test, using only the standard errors of the mean calculated from both eval scores.

However, a two-sample test ignores the hidden structure inside eval data. Since the question list is shared across models, conducting a paired-differences test lets us eliminate the variance in question difficulty and focus on the variance in responses. In our paper, we show how the result of a paired-differences test will be related to the Pearson correlation coefficient between two models’ question scores. When the correlation coefficient is higher, the standard error of the mean difference will be smaller.

In practice, we find the correlation of question scores on popular evals between frontier models to be substantial—between 0.3 and 0.7 on a scale of −1 to +1. Put another way, frontier models have an overall tendency to get the same questions right and wrong. Paired-difference analysis thus represents a “free” variance reduction technique that is very well suited for AI model evals. Therefore, in the interest of extracting the clearest signal from the data, our paper recommends reporting pairwise information—mean differences, standard errors, confidence intervals, and correlations—whenever two or more models are being compared.

Recommendation #5: Use power analysis

The flip side of the statistical significance coin is statistical power, which is the ability of a statistical test to detect a difference between two models, assuming such a difference exists. If an eval doesn't have very many questions, confidence intervals associated with any statistical tests will tend to be wide. This means that models will need to have a large underlying difference in capabilities in order to register a statistically significant result—and that small differences will likely go undetected. Power analysis refers to the mathematical relationship between observation count, statistical power, the false positive rate, and the effect size of interest.

In our paper, we show how to apply concepts from power analysis to evals. Specifically, we show researchers how to formulate a hypothesis (such as *Model A outperforms Model B by 3 percentage points*) and calculate the number of questions that an eval should have in order to test this hypothesis against the null hypothesis (such as *Model A and Model B are tied*).

We believe that power analysis will prove helpful to researchers in a number of situations. Our power formula will inform evaluators of models about the number of times to re-sample answers from questions (see Recommendation #3 above), as well as the number of questions that may be included in a random subsample while retaining the desired power properties. Researchers might use the power formula to conclude that an eval with a limited number of available questions is not worth running on a particular pair of models. Developers of new evals may wish to use the formula to help decide how many questions to include.

Conclusion

Statistics is the science of measurement in the presence of noise. Evals present a number of practical challenges, and a true science of evals remains underdeveloped. Statistics can only form one aspect of a science of evals—but a critical one, as an empirical science is only as good as its measuring tools. We hope that the recommendations in our paper **Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations** will help AI researchers calculate, interpret, and communicate eval numbers with greater precision and clarity than before—and we encourage researchers in the AI community to explore other techniques from experiment design so that they may understand more exactly all the things that they want to measure.

Big tech engineers need big egos

SEANGOEDECKE.COM RSS FEED · 14 MAR 2026 · SOURCE

It's a common position among software engineers that big egos have no place in tech[1]. This is understandable - we've all worked with some insufferably overconfident engineers who needed their egos checked - but I don't think it's correct. In fact, I don't know if it's possible to survive as a software engineer in a large tech company without some kind of big ego.

However, it's more complicated than "big egos make good engineers". The most effective engineers I've worked with are simultaneously high-ego in some situations and surprisingly low-ego in others. What's going on there?

Engineers need ego to work in large codebases

Software engineering is shockingly humbling, even for experienced engineers. There's a reason this joke is so popular:



The minute-to-minute experience of working as a software engineer is dominated by *not knowing things* and *getting things wrong*. Every time you sit down and write a piece of code, it will have several things wrong with it: some silly things, like missing semicolons, and often some major things, like bugs in the core logic. We spend most of our time fixing our own stupid mistakes.

On top of that, even when we've been working on a system for years, we still don't know that much about it. I wrote about this at length in *Nobody knows how large software products work*, but the reason is that big codebases are just that complicated. You simply can't confidently answer questions about them without going and doing some research, even if you're the one who wrote the code.

When you have to build something new or fix a tricky problem, it can often feel straight-up impossible to begin, because good software engineers know just how ignorant they are and just how complex the system is. You just have to throw yourself into the blank sea of millions of lines of code and start wildly casting around to try and get your bearings.

Software engineers need the kind of ego that can stand up to this environment. In particular, they need to have a firm belief that they *can* figure it out, no matter how opaque the problem seems; that if they just keep trying, they can break through to the pleasant (though always temporary) state of affairs where they understand the system and can see at a glance how bugs can be fixed and new features added[2].

Engineers need ego to work in big tech companies

What about the non-technical aspects of the job? Nobody likes working with a big ego, right? Wrong. Every great software engineer I've worked with in big tech companies has had a big ego - though as I'll say below, in some ways these engineers were surprisingly low-ego.

You need a big ego to take positions. Engineers love being non-committal about technical questions, because they're so hard to answer and there's often a plausible case for either side. However, as I keep saying, engineers have a duty to take clear positions on unclear technical topics, because the alternative is a non-technical decision maker (who knows even less) just taking their best guess. It's scary to make an educated guess! You know exactly all the reasons you might be wrong. But you have to do it anyway, and ego helps *a lot* with that.

You need a big ego to be willing to make enemies. Getting things done in a large organization means making some people angry. Of course, if you're making lots of people angry, you're probably screwing up: being too confrontational or making obviously bad decisions. But if you're making a large change and one or two people are angry, that's just life. In big tech companies, any big technical decision will affect a few hundred engineers, and one of them is bound to be unhappy about it. You can't be so conflict-averse that you let that stop you from doing it, if you believe it's the right decision. In other words, you have to have the confidence to believe that you're right and they're wrong, even though technical decisions always involve unclear tradeoffs and it's impossible to get absolute certainty.

You need a big ego to correct incorrect or unclear claims. When I was still in the philosophy world, the Australian logician Graham Priest had a reputation for putting his hand up and stopping presentations when he didn't understand something that was said, and only allowing the seminar to continue when he felt like he understood. From his perspective, this wasn't rude: after all, if *he* couldn't understand it, the rest of the audience probably couldn't either, and so he was doing them a favor by forcing a more clear explanation from the speaker.

This is obviously a sign of a big ego. It's also a trait that you need in a large tech company. People often nod and smile their way past incorrect technical claims, even when they suspect they might be wrong - assuming that they've just misunderstood and that somebody else will correct it, if it's truly wrong. **If you are the most senior engineer in the room, correcting these claims is your job.**

If everyone in the room is so pro-social and low-ego that they go along to get along, decisions will get made based on flatly incorrect technical assumptions, projects will get funded that are impossible to complete, and engineers will burn weeks or months of their careers vainly trying to make these projects work. You have to have a big enough ego to think "actually, I think I'm right and everyone in this room is confused", even when the room is full of directors and VPs.

Sometimes you need to put your ego aside

All of this selects for some pretty high-ego engineers. But in order to actually *succeed* in these roles in large tech companies, you need to have a surprisingly low ego at times. **I think this is why *really* effective big tech engineers are so rare: because it requires such a delicate balance between confidence and diffidence.**

To be an effective engineer, you need to have a towering confidence in your own ability to solve problems and make decisions, even when people disagree. But you also need to be willing to instantly subordinate your ego to the organization, when it asks you to. At the end of the day, your job - the reason the company pays you - is to execute on your boss's and your boss's boss's plans, whether you agree with them or not.

Competent software engineers are allowed quite a lot of leeway about *how* to implement those plans. However, they're allowed almost no leeway at all about the plans themselves. In my experience, being confused about this is a common cause of burnout[3]. Many software engineers are used to making bold decisions on technical topics and being rewarded for it. Those software engineers then make a bold decision that disagrees with the VP of their organization, get immediately and brutally punished for it, and are confused and hurt.

In fact, **sometimes you just get punished and there's nothing you can do.** This is an unfortunate fact of how large organizations function: even if you do great technical work and build something really useful, you can fall afoul of a political battle fought three levels above your head, and come away with a *worse* reputation for it. Nothing to be done! This can be a hard pill to swallow for the high-ego engineers that tend to lead really useful technical projects.

You also have to be okay with having your projects cancelled at the last minute. It's a very common experience in large tech companies that you're asked to deliver something quickly, you buckle down and get it done, and then right before shipping you're told "actually, let's cancel that, we decided not to do it". This is partly because the decision-making process can be pretty fluid, and partly because many of these asks originate from off-hand comments: the CTO implies that something might be nice in a meeting, the VPs and directors hustle to get it done quickly, and then in the next meeting it becomes clear that the CTO doesn't actually care, so the project is unceremoniously cancelled[4].

Final thoughts

Nobody likes to work with a bully, or with someone who refuses to admit when they're wrong, or with somebody incapable of empathy. But you really do need a strong ego to be an effective software engineer, because software engineering requires you to spend most of your day in a position of uncertainty or confusion. If your ego isn't strong enough to stand up to that - if you don't believe you're good enough to power through - you simply can't do the job.

This is particularly true when it comes to working in a large software company. Many of the tasks you're required to do (particularly if you're a senior or staff engineer) require a healthy ego. However, there's a kind of catch-22 here. If it insults your pride to work on silly projects, or to occasionally "catch a stray bullet" in the organization's political fights, or to have to shelve a project that you worked hard on and is ready to ship, you're too high-ego to be an effective software engineer. But if you can't take firm positions, or if you're too afraid to make enemies, or you're unwilling to speak up and correct people, you're too low-ego.

Engineers who are low-ego in general can't get stuff done, while engineers who are high-ego in general get slapped down by the executives who wield real organizational power. The most successful kind of software engineer is therefore a chameleon: low-ego when dealing with executives, but high-ego when dealing with the rest of the organization[5].

1. What do I mean by "ego", in this context? More or less the colloquial sense of the term: a somewhat irrational self-confidence, a tendency to believe that you're very important, the sense that you're the "main character", that sort of thing

   ↩

2. Why is this "ego", and not just normal confidence? Well, because of just how murky and baffling software problems feel when you start working on them. You really do need a degree of confidence in yourself that feels unreasonable from the inside. It should be

obvious, but I want to explicitly note that you don't *just* need ego: you also have to be technically strong enough to actually succeed when your ego powers you through the initial period of self-doubt.

↩

3. I share the increasingly-common view that burnout is not caused by working too hard, but by hard work unrewarded. That explains why nothing burns you out as hard as being punished for hard work that you expected a reward for.

↩

4. It's more or less exactly this scene from Silicon Valley.

↩

5. This description sounds a bit sociopathic to me. But, on reflection, it's fairly unsurprising that competent sociopaths do well in large organizations. Whether that kind of behavior is worth emulating or worth avoiding is up to you, I suppose.

↩

Zen of AI Coding

YOAV AVIRAM · 08 MAR 2026 · SOURCE

Dedicated to all those who are sceptical about the significance of agentic coding, and to those who are not, and are wondering what it means for the future of their profession. The title is an homage to Zen of Python by Tim Peters. Unlike Tim, I am not a zen master. My only aim is to take stock of where we are and where we might be heading. I have been building with coding agents daily for the past year, and I also help teams adopt them without losing reliability or security.

1. Software development is dead

2. Code is cheap

3. Refactoring easy

4. So is repaying technical debt
5. All bugs are shallow
6. Create tight feedback loops
7. Any stack is *your* stack
8. Agents are not just for coding
9. The context bottleneck is in your head
10. Build for a changing world
11. When considering Buy vs. Build, the answer, more often, is build
12. Fast rubbish is still rubbish
13. Software is a liability, a product is an asset
14. Moats are more expensive
15. Build for agents
16. Anticipate modes of failure

Software development is dead

You do not need to write another line of code if you do not want to. Coding agents can accomplish most coding tasks with the right direction.

Software development, as the act of manually producing code, is dying. A new discipline is being born. Your role in it is vital, but it is no longer centered on typing code. It is centered on framing problems, shaping context, defining constraints, and judging outcomes.

The marginal cost of code is collapsing. That single fact changes everything that follows.

Code is cheap

The economics of software have changed.

When coding is cheap, implementation stops being the constraint. You can build ten things in parallel. You cannot decide, validate, and ship ten things in parallel, at least not without changing the rest of the pipeline.

Cost of delay shifts. It is no longer about developer days. It is about time stuck in other bottlenecks: product decisions, unclear requirements, security review, user testing, release processes, and operational risk. Agents can flood these queues. Inventory grows. Lead time grows. Delay becomes more expensive, not less.

This changes prioritization. The highest leverage work is what unblocks shipping: tight feedback loops, tests, evals, guardrails, observability, and clear acceptance criteria. Anything that turns “we can build it” into “we can trust it”.

Agents can help alleviate some of these bottlenecks. The challenge is applying them outside of coding (see Agents are not just for coding)

Refactoring is easy

The cost of changing your mind is lower than it has ever been. Architectural decisions that once felt permanent are now provisional.

You chose React. Two months later, you regret it. Ask an agent to rewrite the project.

Making imperfect decisions is no longer fatal. In fact, it can be productive. A flawed reference implementation provides better context than a pristine specification. Agents reason more effectively from concrete artifacts than from abstract intent.

Rapid iteration is now the default mode. Over the last three months, we rebuilt our CMS four times from scratch, each with a different architecture. Each time we learned more about what we actually needed and what works.

When code is cheap, you can run more small bets.

So is repaying technical debt

There is little excuse for stale dependencies or ignored security patches. Updating libraries, migrating APIs, modernizing patterns. These are prompts away.

Prune your code regularly. Upgrade aggressively. Keep the surface clean.

Technical debt has not disappeared. But the cost of servicing it has dropped. That changes the incentive structure. Neglect becomes less defensible.

All bugs are shallow

Linus's law states that "given enough eyeballs, all bugs are shallow." Those eyeballs are now abundant.

Ask one model to review your code. Ask another. They may find different issues. Ask them to fix what they find. Often they will succeed.

How "dense" are bugs given a piece of code is a philosophical question. Bugs are not magically gone. Agents do not achieve perfection. From a practical perspective, however, the limiting factor is no longer the number of reviewers. It is the tightness of your feedback loops.

The claim here is profound: comprehension of the codebase at the function level is no longer necessary. At the same time, relying entirely on the models is a recipe for disaster. You still have a job! (see: create tight feedback loops, anticipate modes of failure)

Agents can sometimes one-shot a task.

We've built agents that create a complete custom-made marketing website in one shot, but this is by far the exception. In most cases, agents operate best when iterating towards a well-defined goal, guided by feedback.

Good tests are the first feedback loop. The agent will iterate until the tests pass. It is your responsibility to make sure the tests are adequate (by instructing the agent to create good ones) and to make sure the agent does not cheat (by weakening the tests or simply skipping them).

Give your agent access to your CI to create a second feedback loop. Give it access to your server logs to create a third. Giving the agent away to visually inspect a UI is yet another example.

Your job is to design environments where iteration converges toward correctness instead of drifting toward plausible nonsense.

Velocity without feedback is chaos.

Any stack is your stack

Agents are competent across most major stacks where training data exists. Since you are not writing the code, your attachment to a specific stack becomes less relevant.

Do not limit yourself to what you personally know. Your conceptual understanding transfers more than you think.

When you lack terminology or domain knowledge, ask the agent to brief you. The barrier to entry into unfamiliar ecosystems is lower than it has ever been.

Agents are not just for coding

Coding is only the beginning.

Agents can assist with business analysis, UX, infrastructure, operations, ad campaigns, analytics configuration, even accounting workflows.

I migrated four WordPress blogs from an overpriced host to Hetzner in minutes after postponing the task for years. The blocker was never a technical difficulty. It was attention. Claude completed the task in 15 minutes.

Agents reduce the activation energy required to execute tedious but valuable work.

Grant access carefully. Limit scope. Audit results. Direction without oversight is reckless.

The context bottleneck is in your head

Context engineering has improved. Tooling has improved.

Yet when running multiple agents in parallel, the real bottleneck is human coordination.

You must track what each agent is doing. You must remember which assumptions were made. You know what questions to ask next.

As agents become better at managing their own context, your ability to supervise them becomes the limiting factor.

The constraint is no longer tokens. It is cognition.

Build for a changing world

New models. New capabilities. New frameworks. New skills.

The ground shifts daily. Some models are better at reasoning. Some at coding. Some at translation.

Flexibility is not optional. It must be built into your workflow and products. Both must be engineered for experimentation and adaptability.

When considering Buy vs. Build, the answer, more often, is build

When code was expensive, buying made sense. When code approaches zero marginal cost, the calculus changes.

Every paid API is now a question. Could we replicate this internally? Should we?

For our QA agent, we needed full-page screenshots of live sites. Many APIs offered this. Instead, we deployed Playwright to AWS Lambda, tailored to our needs, and hardened.

This is not a blanket argument against SaaS. Maintenance, security, and scale still matter. But many small and medium services that once required vendors are now within reach.

Some call this the end of SaaS.

Fast rubbish is still rubbish

Speed is intoxicating. It is also dangerous.

You can generate enormous amounts of code quickly. But velocity and lines of code written were always terrible indicators of progress.

Refactoring is cheap, but it is cheaper to detect flawed direction early.

Discipline matters more, not less, when execution is frictionless.

Software is a liability, a product is an asset

Software costs money to maintain. It only becomes an asset when it produces value for someone.

This was always true. It is harder to remember now because building is so easy.

It is tricker then ever to resist the temptation to add features. Resist it. Build what is used. Kill what is not.

Feature gaps close quickly. What once took years to replicate can now take months. What took months can now take days.

A feature set is no longer a durable moat.

Moats shift toward distribution, data, brand, network effects, regulation, and ecosystem integration.

The barrier to entry falls. The barrier to defensibility rises.

There is a new hierarchy.

At the base are models (Opus 4.6, GPT-5.3-codex). On top of them sit agents (Claude Code, OpenClaw). On top of agents sit services.

Today, agents use services designed for humans. It works, but it is inefficient. The next step is services designed for agents (Moltbook being the harbinger).

Make your services accessible to agents. Expose structured context. Provide machine-readable surfaces.

Agentic commerce is emerging. In some cases, a markdown endpoint is enough. In others, the service itself must be designed agent-first.

Agent-first is not a tooling choice. It is a product decision. It changes your interfaces, your reliability model, your metrics, and what you build next.

We are building an agent-first CMS. If we succeed, humans will not need to use it, though they can.

AX is the new UX

Do not assume competence implies perfection.

Like an engineer overseeing the construction of a bridge, your job is not to lay bricks. It is to ensure the structure does not collapse.

Agents will make mistakes. They will drift off course. They will introduce subtle flaws. They will create security vulnerabilities. One of mine attempted to commit a .env file to git. They can be inventive in how they self-sabotage.

Be one step ahead. Ask yourself how this could break. Where could it leak. What could it expose. What assumptions is the agent quietly making.

Play the what-if game relentlessly. Then build guardrails.

Automate checks. Lock down permissions. Isolate environments. Add monitoring. Create recovery paths. Make rollback trivial.

Remember, code is cheap, and you have new superpowers. Build tools to allow you to probe deep, discover vulnerabilities, and test for anticipated failure modes. Design systems that expect failure and degrade gracefully.

This is the distinction between software development and software engineering.

I work with engineering and product teams adopting agents. The goal is simple: ship faster without lowering standards.

I usually help in two ways:

Agent-first software engineering
We turn agents into part of the system, not a novelty. We design workflows, guardrails, and supporting tools so the team can delegate confidently.

If you want to operationalize coding agents without turning your codebase into a casino, get in touch.

Agent-first product strategy
I help companies shift products from human-centric interfaces to agent-accessible, and where it makes sense, agents-first. That means clarifying what agents should be able to do, what constraints they must operate under, and what needs to change in your APIs, permissions, reliability model, and product surface area to support them.

This typically results in an agent-access roadmap, an operating model, and a prioritized set of product changes that make agents a first-class user.

I'm the co-founder of WhiteBoar, where AI agents build your brand and launch visually striking marketing websites, complete with professional multi-lingual content, in minutes, not months. Contact me at yoav@whiteboar.it.